Quantifying Process Model Generalization: A Generative N-gram Metric and a Cross-Paradigm Benchmark*

Christian Daniel Krengel and Tianhao Geng

Ludwig-Maximilians-Universität München, Munich, Germany
...

Abstract. Generalization is the ability of a process model to accept behavior that is valid under the underlying process but absent from the event log. It is the least understood of the four established process-model quality dimensions. Published generalization metrics are non-commensurable, are rarely compared against one another and are almost never validated against a ground truth on real-life logs. This report makes four contributions. First, we argue for a *strict construct definition*: a generalization metric should measure only the True-Positive of future valid behavior: measuring False-Positives is the task of precision, and conflating the two destroys the diagnostic value of the quality-dimension quadrant. We use *trace model* and *flower model* as two extreme cases in order to serve as a theoretical test for the metrics and a purity litmus. Second, we present *ShadowGen*, a generative metric that derives a stochastic trace generator from the log using variable-order N-gram statistics with Katz-style backoff and Good-Turing novelty estimation, generates a synthetic shadow log of unseen traces and scores a model by replaying it under two complementary readings (graded fitness and strict acceptance). Third, we design a benchmark which compares our metric and its development history and seven published baselines against eight discovery configurations, anchored by variant-based cross-validation as ground truth and assessed on four criteria: correlation, ranking, absolute error, and discriminative spread. Fourth, we evaluate on real-life event logs of differing representativeness. On the logs (to date), the current metric tracks the ground truth at Pearson 0.996/0.994 (mean absolute error 0.023/0.019) at interactive runtime, whereas the field's default metric (PM4Py) does not. It correlates negatively on one log and fails the flower-model litmus on every log. Two published metrics are computationally infeasible on a 1,050-case log; an iterative version study attributes a 3× calibration improvement to two measured defects; and a strict acceptance reading, validated against an acceptance-based ground truth, anchors both construct poles exactly. [insert D3–D5 here]

Keywords: Process mining · Process discovery · Generalization · Model quality · Conformance checking · Benchmark.

* Report, M.Sc. Computer Science, LMU Munich.

1 Introduction

1.1 Problem statement and motivation

Process discovery algorithms construct process models from event logs recorded by information systems [1]. A discovered model is conventionally assessed along four quality dimensions [7]: *fitness* (the model allows observed valid behavior), *precision* (it does not allow invalid behaviors), *simplicity* (it is structurally sparse) and *generalization* (it allows future valid behavior). An event log is only a sample of an underlying process. Generalization captures whether a model describes the process rather than memorizing the sample.

Generalization is the most elusive of the four dimensions for a simple reason: it attempts to make a claim about behavior that, by definition, is not in the data. The literature offers various metrics, but they approach the problem from fundamentally different angles: counting rarely used model parts [7], information-theoretic relevance of stochastic models [4], adversarial anti-alignments [8], generative adversarial networks [22], statistical bootstrapping [18] and biodiversity-inspired species estimation [12]. These different paradigms produce different scores that are not clearly commensurable. Prior work has compared such metrics to one another and found that, unlike fitness and precision, generalization measures do not agree [11], and has assessed conformance measures against axiomatic propositions [2,21]. What we provide is a comparison across the various paradigms validated against an empirical, log-derived generalization ground truth (variant-based k-fold) on common real-life logs, which alone can establish which metric, if any, tracks unseen-but-real behavior. The practical consequence: no validated consensus generalization metric exists. The de-facto default, the generalization measure shipped in PM4Py [6], is widely used but, as we show, does not track empirical generalization. A practitioner who wants to know whether a discovered model will accept tomorrow's cases is therefore left without a validated, affordable instrument.

1.2 Proposed solution

We pursue two goals. The first is a *new metric*. *ShadowGen*[1] learns a variable-order Markov model of the log: N-grams up to order N=6 with Katz-style back-off [14]. From this, Good-Turing estimation [9] gives a per-context probability that the next event is new. ShadowGen then generates a synthetic *shadow log* of plausible-but-unseen traces and scores a model by how well it replays them [20]. The second goal is a *validated benchmark*. We test the metric, its past iterations and seven published baselines on eight discovery configurations, and validate each metric against variant-based hold-out (both k-fold cross-validation and leave-one-variant-out) which measures acceptance of unseen-but-real behavior directly.

[1] The implementation package keeps the legacy name `HybridGen`, from an earlier design that paired the generative score with a structural term. That term was retired (Sect. 4.2), so the metric here is the pure generative component.

1.3 Summary of key results

On the logs evaluated (to date), ShadowGen reproduces the cross-validation ranking and is calibrated to it (Pearson 0.996/0.994, mean absolute error 0.023/0.019 on D1/D2). The same agreement also holds under leave-one-variant-out hold-out, at about 2 to 7 seconds per model.

The resampling-based bootstrap is the only baseline of comparable quality. The widely used PM4Py metric does not track the ground truth: it correlates negatively on D1 and fails the flower-model litmus on every log. Two published metrics failed to return a result in a reasonable time on a 1,050-case log and are reported as infeasible. A version study attributes the calibration (a 3× error reduction) to two specific, measured fixes, not to richer context. Finally, a strict acceptance reading of the same shadow log, validated against an acceptance-based ground truth (Pearson 0.998/0.997), anchors the construct's two poles at exactly 0 and 1. [insert D3–D5 here]

1.4 Positioning within process mining

This work sits between conformance checking and model-quality evaluation. Most prior generalization metrics propose a measure and demonstrate it qualitatively. Our stance is instead validation: a single-shot score is trustworthy only if it agrees with a direct, expensive measurement of generalization (hold-out acceptance). We also adopt a deliberately narrow definition of generalization: focusing only on the acceptance/fitness of future valid behavior, separate from precision. This allows us to turn the otherwise-useless flower model into a diagnostic litmus test. Finally, any metric and any hold-out ground truth is computed from a log, so its accuracy is bounded by how representatively the log samples the system [13]. This bound applies to every log-based analysis, not just ours, so we treat representativeness as a property of the data and span it deliberately across our datasets. Detailed positioning is deferred to Sect. 3.

1.5 Scope

We evaluate on two real-life logs so far, D1 (Sepsis) and D2 (BPI 2013 Incidents), with the same battery of methods on each. Three larger logs (D3–D5) are in progress (Sect. 7). Some external baselines cannot be run as published. We either adapt them to a shared protocol or report them as infeasible, and we state every such case explicitly (Sects. 5.4, 6.2).

1.6 Structure

Section 2 fixes notation and the process-mining concepts the method builds on. Section 3 surveys related metrics by theme and states the gap. Section 4 presents the construct, the metric, its pseudocode, a running example, and a complexity analysis. Section 5 describes the implementation and where it deviates from the formal method. Section 6 reports the benchmark: setup, feasibility, cross-paradigm comparison, version study, acceptance reading, runtime, discussion, and threats. Section 7 concludes.

2 Background

Event logs. Let $\mathcal{A}$ be a finite set of activities. A trace $\sigma = \langle a_1, \ldots, a_k \rangle \in \mathcal{A}^*$ is a finite sequence of activities; an event log L is a multiset of traces. Traces with an identical activity sequence form a *variant*; we write $V(L)$ for the set of variants and $\#\sigma$ for the case count of variant σ. Real logs are typically dominated by a few frequent variants and a long tail of rare ones. We summarize a log's repetitiveness by $\mathrm{TLRA} = 1 - |V(L)|/|L|$, the empirical probability that a new case repeats a known variant.

Process models and discovery. A process discovery algorithm (a miner) maps a log to a process model. In this work all models are accepting Petri nets (N, m_i, m_f) discovered with PM4Py [6]; the discovery algorithms we use range from the Alpha miner [3] through the Heuristics miner [23] to the Inductive miner [15,16]. Two extreme constructions serve as theoretical boundaries: a *trace model*, one isolated path per observed variant, and a *flower model*, all activities in a single self-loop accepting every sequence over $\mathcal{A}$.

Token replay and its two readings. Token-based replay [20] replays a trace on a net, firing transitions and accounting for produced, consumed, missing, and remaining tokens. It yields two distinct quantities that this report keeps separate:

- a graded trace fitness in $[0, 1]$ derived from the token bookkeeping, which awards partial credit (a trace the net cannot actually execute end-to-end can still score, e.g., 0.7);
- a boolean perfect-replay flag (`trace_is_fit`): the trace replays with no missing or remaining tokens, i.e., it is in the model's language.

The fitness of a log is the mean trace fitness; the acceptance rate is the fraction of perfectly replayed traces. As Sect. 6.6 shows, the graded and boolean readings can diverge sharply, and the gap between them is itself informative.

Variable-order language models. The generator at the core of our metric is a variable-order Markov (N-gram) model of the log. Two classical techniques from statistical language modeling are reused. *Katz backoff* [14] predicts the next symbol from the longest recent context that has sufficient statistical support, falling back to shorter contexts when a long context is too sparse to trust. *Good–Turing estimation* [9] estimates the total probability mass of unseen events from the number of events observed exactly once (singletons): if a fraction N_1/N of observations were one-offs, roughly that fraction of future observations should be expected to be novel.

3 Related Work

We group existing approaches by paradigm. Method identifiers (M2–M8) anticipate the benchmark roles of Sect. 6.

Structural counting (M2). The PM4Py built-in generalization metric [7,6] penalizes models whose internal parts are visited rarely while replaying the observed log: a model in which every node is used often is deemed general. It is computationally trivial but never confronts the model with unseen behavior, and, as our results show, it does not track held-out acceptance.

Entropy-based (M3). Entropic relevance [4] measures the average number of bits needed to encode the log under a *stochastic* model (a stochastic deterministic finite automaton). It unifies fitness- and precision-like aspects in one information-theoretic quantity, but requires a stochastic model; a plain Petri net must first be annotated with transition probabilities, which is a non-trivial conversion.

Adversarial (M4). Anti-alignments [8] search for model traces maximally distant from the log: a model able to produce behavior far from everything observed is judged over-general. The underlying optimization is exponential in nature, which makes the published implementation infeasible on real-life logs (Sect. 6.2).

Generative / neural (M5). AVATAR [22] trains a sequence GAN to approximate the system that produced the log, samples unseen-but-plausible traces from the generator, and scores generalization as the harmonic mean of fitness and precision against those samples. AVATAR is conceptually closest to our metric (both are generative) but replaces an interpretable statistical generator with a neural one at a cost of hours of GPU training per log, and its harmonic-mean score is an F-measure that blends precision into the result (see the construct discussion in Sect. 4.1).

Resampling / bootstrap (M6). Bootstrap generalization [18] treats the log as a sample from an unknown trace population, repeatedly resamples it and "breeds" recombined traces via genetic crossover, and aggregates model quality over the replicates with confidence intervals. Its target quantity is model–system recall (the fraction of system behavior the model accepts), which coincides with our construct (Sect. 4.1). It is the strongest baseline in our study. Crucially, its test traces are recombinations of observed material; it does not model or inject genuinely novel events in a principled way.

Species / diversity (M7). SpeciAL [12] transfers species-richness estimation from ecology to event data: activities or N-grams are "species," and completeness/coverage profiles quantify how representative a log (or a simulated log) is. We use the coverage ratio between model-simulated and original logs as a generalization proxy.

Pattern-based (M8). Reißner et al. [19] read generalization off automata-based behavioral patterns; the available implementation proved unusable on our logs (Sect. 6.2).

Hold-out as empirical ground truth. The most direct operationalization of generalization is hold-out evaluation: discover a model on a subset of variants, then measure how well withheld variants replay. Variant-based k-fold cross-validation and leave-one-variant-out realize this at increasing granularity. They are expensive (leave-one-variant-out requires one discovery per variant) and, importantly, are only defined when the model can be rediscovered from log subsets, so they evaluate a discovery algorithm rather than a model artifact in hand. These properties make them ideal as a validation reference but impractical as a routine metric.

Membership versus graded semantics. The textbook definitions of generalization are membership-shaped: "the likelihood that previously unseen but valid behavior is allowed by the model" [1], a yes/no question per trace. The prevailing practice, by contrast, is graded, because token- and alignment-based fitness were introduced precisely so that an almost-correct trace does not count the same as noise [20]. Our metric reports both readings of the same shadow log and validates each against its matching ground truth.

Comparative and axiomatic evaluation. A few studies assess the metrics themselves rather than models. Janssenswillen et al. [11] ran a large-scale comparison of fitness, precision, and generalization metrics and found that, unlike the other two dimensions, generalization metrics *do not agree with one another.* A follow-up tested whether such measures capture model–system quality on synthetic logs with a known ground-truth system, and concluded that it is "nearly impossible to objectively measure the ability of a model to represent the system" [10]. A second strand evaluates conformance measures against *axioms*: van der Aalst's conformance propositions [2] and Syring et al.'s check of which measures satisfy them [21]. A separate line benchmarks process *discovery algorithms* using these metrics as given [5], not the metrics themselves. None of these compares a cross-paradigm set of generalization metrics against an empirical hold-out reference on real logs.

The gap we fill. Three gaps recur across these groups. (i) *No cross-paradigm validation.* To our knowledge, no prior work compares generalization metrics across paradigms against an empirical, log-derived hold-out ground truth on common real logs; the closest attempts compare metrics only to one another [11] or use synthetic known-system data [10]. It is therefore unknown which paradigm actually tracks held-out acceptance. (ii) *Opacity.* The most faithful baselines (neural, bootstrap) return a number with little insight into why a model scored as it did. (iii) *Construct drift.* Several "generalization" metrics quietly fold in precision, which conflates two orthogonal failure modes. Our contribution is a metric that is (a) validated against cross-validation ground truth and shown to track it, (b) interpretable, exposing per-context novelty, mutation flags, an openness profile, and a strict acceptance reading, and (c) construct-pure by

design, with a benchmark that makes construct drift visible through the flower-model litmus.

4 Method

4.1 Construct: what generalization is and is not

We adopt a strict construct: **generalization is the degree to which a model accepts future, valid behavior of the underlying process that is absent from the log, and nothing else.** Generalization is *not* precision. Detecting over-permissiveness (acceptance of *invalid* behavior) is the precision dimension's task. Folding a permissiveness penalty into a generalization score makes any mid-range value ambiguous: one cannot tell "rejects future valid behavior" from "accepts invalid behavior." That destroys the diagnostic value of the quality quadrant. One does not redefine recall because a degenerate classifier attains maximal recall; one reports precision next to it. Metrics that blend the two (for example, AVATAR's harmonic mean of fitness and precision) measure a legitimate but *different* construct and must not be read as pure generalization.

Two degenerate models operationalize the poles. The *flower model* accepts every sequence over $\mathcal{A}$; its generalization is maximal *by definition*, and it is a useless model because of its precision and simplicity, not its generalization. A pure generalization metric must therefore score it ≈ 1. The *trace model* memorizes observed variants and accepts nothing unseen; it is the 0 pole. This yields a *construct-purity litmus*: a metric that scores the flower model below 1 demonstrably mixes precision or structure into its score. The metric is meant to be read *alongside* precision: a model with $Gen \approx 1$ and precision ≈ 0 is diagnosed as over-permissive by the precision column, exactly where that information belongs.

4.2 Framework

ShadowGen scores a model by the replay quality of a synthetic *shadow log*: traces that are plausible under the log's behavior but, by construction, absent from it. The generator is derived from the log alone, and the model under evaluation is consulted only at replay time (Fig. 1). This keeps the generation independent of the artifact being judged. An earlier design paired this generative score with a structural term that penalized rarely-used model parts; we dropped it, because its anti-flower role belongs to precision (Sect. 4.1) and its anti-overfitting role is already covered by replay failures of recombined traces. The current score is therefore the pure generative component.

4.3 Shadow log generation

Variant statistics. The log is compressed to its variants with case counts; every count below is weighted by how often the variant occurs ($\#\sigma$). For each order $n \in \{1, \ldots, N\}$ (default N=6) and each observed context s (a length-n

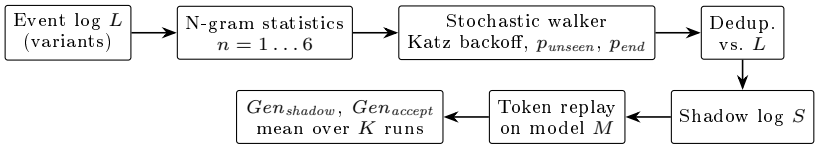

Fig. 1. ShadowGen pipeline (current version). The generator is derived from the log only; the model under evaluation enters only at the replay step.

activity sequence), we record the successor multiset $C_n(s)$, where $C_n(s)[a]$ is the frequency with which activity a follows s, and separately the termination counts: $T_n(s)$, the number of occurrences of s, and $E_n(s)$, the number of those occurrences at the end of a trace.

Katz backoff. At generation time the walker holds a partial trace σ and resolves its decision context to the deepest order with adequate support. Let τ be a support threshold (default 5). The *resolved order* is the largest $n \leq N$ such that the suffix $s = \text{suffix}_n(\sigma)$ satisfies $|C_n(s)|_1 := \sum_a C_n(s)[a] \geq \tau$ *and* the Good–Turing mass below is < 1; otherwise n is reduced, falling back ultimately to n=1. Thus N is an upper bound, not a fixed operating point: on sparse logs the effective order is reduced naturally.

Good–Turing novelty. For the resolved context s, the probability that the next activity is one never observed after s is estimated as

$$p_{unseen}(s) \;=\; \frac{N_1(s)}{|C(s)|_1}, \qquad N_1(s) = |\{a : C(s)[a] = 1\}|, \tag{1}$$

i.e., the number of singleton successors over the total successor count. With probability $p_{unseen}(s)$ the walker *mutates*; otherwise it *exploits*.

Mutation proposal (Katz-consistent). When a mutation fires, the inserted activity must be novel in the resolved context yet plausible. We draw it from the deepest *shorter* context that offers a successor unseen at the resolved order. Let R be the resolved order and $\mathit{seen} = \{a : C_R(s)[a] > 0\}$. We scan $m = \min(|\sigma|, N), \ldots, 1$ and take the first m for which $\{a : C_m(\text{suffix}_m(\sigma))[a] > 0\} \setminus \mathit{seen} \neq \emptyset$, sampling from that residual set; if none exists we fall back to global activity frequencies restricted to $\mathcal{A} \setminus \mathit{seen}$. A short observation fixes the search direction: a longer context's successor set is always contained in its suffix's, since every occurrence of s is also an occurrence of its suffix s' with the same next activity. Contexts of order $\geq R$ therefore add nothing new (orders $> R$ are subsets of the resolved successors, and order R equals *seen*), so the proposal always finds its candidates at order $R{-}1$ or shorter, one level below the resolved order. This is the v2.5 correction over earlier versions, which drew the mutated activity uniformly from $\mathcal{A}$ and thus injected behavior that was often process-impossible at that position.

Exploitation: two sampling modes. For a non-mutation step the walker samples a seen successor a with weight $w(C(s)[a])$, normalized over the successor set. We support two weighting modes:

$$w_{\text{mle}}(c) = c \qquad \text{vs.} \qquad w_{\log}(c) = \ln(c+1). \tag{2}$$

The `mle` mode samples from the maximum-likelihood estimate of the next-activity distribution (proportional to observed frequency), so the shadow log mirrors the *expected* future. The `log` mode deliberately flattens the distribution, up-weighting rare paths; it is a rare-behavior *stress test*. The same weighting governs the mutation-proposal sampling, but never the mutation *rate* of Eq. (1), which always uses raw counts.

Context-aware termination. Whether the walk ends after context s is decided by a termination probability resolved with the same backoff:

$$p_{end}(s) \;=\; \frac{E(s)}{T(s)}. \tag{3}$$

Conditioning termination on the n-gram context (rather than the last activity alone) fixes premature termination when activities that legitimately end traces appear early in a generated trace.

Deduplication and scoring. Each generated trace is compared against the set of original variants and regenerated if it is a copy (up to a retry cap), so the shadow log contains only unseen sequences. A shadow log S of $\min(1000, |L|)$ traces is replayed on the model; generation and replay are repeated K=5 times (all randomness seeded). We report two scores: Gen_{shadow} (mean graded trace fitness, the headline) and Gen_{accept} (fraction of perfectly replayed traces, the strict reading). We also report each score separately for shadow traces that contain a mutation and those that do not, which shows how a model handles genuinely novel events versus mere recombinations.

4.4 Algorithm

Algorithm 1 gives the generation procedure; statistics gathering and scoring are described above.

4.5 Running example

Figure 2 traces one mutation on a synthetic Sepsis pathway (seed 42). Backoff does two jobs. It leaves the over-sparse order-6 context (seen five times, every successor once, so $p_{unseen} = 1$) for order 5 to set *how often* to invent an event ($p_{unseen} = 1/35 \approx 3\%$); when the draw fires, it keeps backing off to order 4 to choose *what* to invent (Release-D). The inserted activity is novel in the exact context, yet attested in a related shorter one, and the resulting trace (absent from the log) is flagged as a mutated shadow trace.

Algorithm 1 Shadow-log generation (one iteration)

Require: log L, order N, threshold τ, count T, cap M, weighting w

build C_n, T_n, E_n for $n = 1..N$ from variants of L; collect starts, alphabet, variant set $\mathcal{V}$
$S \leftarrow \emptyset$
for $i = 1$ to T **do**
  repeat
    $\sigma \leftarrow \langle$ sample start activity $\rangle$; *mut* $\leftarrow$ false
    while $|\sigma| < M$ **do**
      resolve context s by Katz backoff (τ); compute $p_{unseen}(s), p_{end}(s)$
      if *rand*() $< p_{end}(s)$ **then break**
      end if
      if *rand*() $< p_{unseen}(s)$ **and** a novel candidate exists **then**
        append a Katz-consistent novel activity (weight w); *mut* $\leftarrow$ true
      else
        append a seen successor sampled $\propto w(\cdot)$; **break** if dead end
      end if
    end while
  until $\sigma \notin \mathcal{V}$ **or** retry cap reached
  $S \leftarrow S \cup \{(\sigma, mut)\}$
end for
return S

4.6 Time and space complexity

Let $|V|$ be the number of variants, $\bar{\ell}$ the mean variant length, $\ell_{\max}$ the longest, $A = |\mathcal{A}|$, N the maximal order, T the shadow-log size, $M \approx 2\ell_{\max}$ the walk cap, K the iteration count, and $|M_P|$ the size of the Petri net under evaluation. Let $E_V = \sum_{\sigma \in V(L)} |\sigma|$ be the total length of distinct variants.

Statistics ($O(N\,E_V)$). Each of E_V positions contributes to N orders, each $O(1)$. Space is $O(N\,D)$ where $D \leq N\,E_V$ is the number of distinct n-grams stored.

Generation ($O(T\,M\,\bar{b})$, on average). Each of T traces takes up to M steps. Per step, context resolution and termination are memoized on the last-N activities, so each distinct context is resolved once at cost $O(N + b + A)$ (backoff scan, building the successor and novel-candidate lists, b the branching factor), and reused thereafter; sampling a successor costs $O(\bar{b})$ with $\bar{b} \leq A$. Deduplication is an $O(1)$ expected set lookup per attempt. Thus generation is $O(T\,M\,\bar{b}) + O(D\,(N + A))$, i.e., effectively linear in the shadow-log size.

Replay ($O(T\,M\,|M_P|)$). Token replay is roughly linear per trace in trace length and net size.

Total. Per evaluated model, $O\big(K\,(N\,E_V + T\,M\,\bar{b} + T\,M\,|M_P|)\big)$. Two observations matter. First, the cost is linear in the log *through its variants*, not its cases: variant compression makes the metric scale with behavioral diversity rather than volume. Second, this contrasts sharply with the leave-one-variant-out ground truth, which performs $|V|$ full model discoveries ($O(|V| \cdot \text{discover})$)

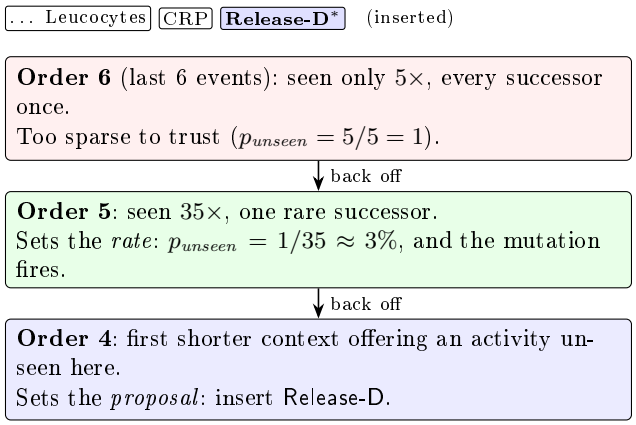

Fig. 2. Katz-consistent mutation (Sect. 4.5). Backoff sets both *how often* to invent an event (the rate, resolved at order 5) and *what* to invent (the proposal, found at order 4): novel in the exact context, yet attested in a related shorter one.

and is therefore orders of magnitude more expensive on variant-rich logs. In the current implementation the statistics are rebuilt once per iteration, contributing the factor K to the $N\,E_V$ term; this is an implementation artifact (Sect. 5.5) rather than an intrinsic cost, as the model could be built once and reused across iterations.

4.7 Assumptions, scope, and limitations of the approach

The method assumes a control-flow view (activity sequences; data, resources, and time are ignored) and a finite activity alphabet. It assumes the log is a representative-enough sample that local N-gram statistics are meaningful; on extremely sparse logs the backoff collapses toward n=1, reducing the metric to a 1-gram directly-follows generator, which remains a reasonable if weaker baseline (Sect. 6.5). The shadow log models future behavior as recombinations of observed local patterns plus a calibrated tail of novel events; behavior driven by long-range or data-dependent dependencies beyond order N is not captured. The metric exposes parameters (N=6, τ=5, K=5), but they are fixed defaults: N and τ are calibrated once (Sect. 6.1) and never tuned per dataset, and Katz backoff makes N an upper bound that adapts downward on sparse logs. In use the metric is therefore as parameter-free as fitness and precision are. Finally, it measures only the generalization construct of Sect. 4.1 and is meant to be read alongside a precision metric, not in isolation.

5 Implementation

5.1 Architecture

The metric is implemented in Python on top of PM4Py [6]. The code is organized as an *algorithm registry*: each metric version registers itself under a name (`v1`, ..., `v26`), and a benchmark runner loads a version by name and calls a uniform `evaluate_miner(log, miner)` entry point. Each version is a *frozen module*. Rather than mutating one file over time, every design stage is preserved as the exact code that produced its benchmark numbers, which is essential for the version study (Sect. 6.5) and for provenance. The benchmark layer comprises model preparation, per-method bridges (including subprocess bridges to external Java/Docker baselines), the ground-truth scripts, and two runners (`run_m1_family.py` for the metric family, `version_comparison.py` for quick multi-seed checks).

5.2 Key design decisions

Variant compression. Statistics and deduplication operate on variants weighted by case count, making preprocessing independent of case volume (Sect. 4.6). **Memoization.** Context resolutions are cached on the last-N activities, turning the per-step cost effectively constant after warm-up. **Iterative walker.** Generation uses an explicit loop rather than recursion, avoiding stack limits on long walks. **Determinism.** All random number generators are seeded (*seed*=42) so that every cell is reproducible. **Provenance.** Every (dataset, miner, method) cell writes a sidecar JSON capturing exact parameters, raw per-iteration scores, runtime, and integrity counters; these files are the single source of truth from which all tables are derived, and a result without a matching config is treated as invalid. **Integrity counters.** The generator reports `duplicates_kept` (deduplication retry cap exhausted) and `truncated_traces` (walks cut at the length cap), so any deviation of the shadow log from its specification is visible rather than silent.

5.3 The tool in use

The benchmark is driven from the command line. Listing 1.1 shows a run of the metric family on one dataset; Listing 1.2 shows a representative result record; Listing 1.3 shows a generated shadow trace with its mutation point (the running example of Sect. 4.5). The tool prints the legacy package name `HybridGen`.

Listing 1.1. Running the metric family over all miners on D1.

```
$ python benchmark/run_m1_family.py --dataset D1
Discovering models (cached across methods):
  Trace_Filtered  751t/377p  ...   Flower 16t/1p
--- M1g: HybridGen v26 (MLE weighting) ---
    Inductive_Strict   0.9838 +- 0.0020   (6.7s)
    Flower             1.0000 +- 0.0000   (3.5s)
SUMMARY  Pearson 0.996  Spearman 1.000  MAE 0.023  Flower 1.000
```

Listing 1.2. Sidecar config JSON for one cell (abridged).

```
{ "dataset": "Sepsis", "miner": "Inductive_Strict", "method": "M1g",
  "seed": 42,
  "parameters": { "algorithm_version": "v26", "max_n": 6,
                  "successor_weighting": "mle", "num_shadow_traces": 1000,
                  "iterations": 5 },
  "results": { "mean": 0.9838, "std": 0.0020, "gen_accept": 0.7444,
               "duplicates_kept": 0, "truncated_traces": 0,
               "runtime_s": 6.7 } }
```

Listing 1.3. A generated shadow trace; the starred event is the Katz-consistent mutation.

```
ER-Reg  ER-Triage  ER-Sepsis-Triage  LacticAcid  CRP  Leucocytes
IV-Antibiotics  Admission  Admission  Leucocytes  CRP  [Release-D]*  Return-ER
```

5.4 Baseline integration realities

Integrating the seven external baselines surfaced engineering realities that materially affect the evaluation and must be reported as such. The PM4Py metric (M2) is a one-line library call. AVATAR (M5) runs in a TensorFlow 1 Docker image and required a greedy longest-match decoder to reconstruct multi-word activity names from the GAN's token output. SpeciAL (M7) is a Python package used directly. Two baselines could not be used as published: anti-alignments (M4) and pattern-based generalization (M8) did not complete on real logs (Sect. 6.2). A third, entropic relevance (M3), requires a Petri-net-to-stochastic-automaton conversion that is not yet implemented, so it currently annotates a log-derived directly-follows graph and returns an identical, non-discriminating value for all models.

Bootstrap (M6) is a construct-faithful adaptation. Polyvyanyy et al. define bootstrap generalization as model–system *recall*: the score is 1 exactly when the model accepts every new trace the system produces [18], the same acceptance construct we use (Sect. 4.1). Our bridge runs the genuine `bsgen` bootstrap-with-breeding *sampler* and scores the bred traces by PM4Py token-replay fitness, i.e. a *graded* model–system recall, the way Gen_{shadow} grades the shadow log. We deliberately avoid the Entropia `-bgen` tool's harmonic mean of model–system precision and recall: that F-measure folds precision back in and is no longer the paper's recall-based generalization. It scores the flower model 0.18 instead of 1.0 on D1, failing the litmus of Sect. 4.1. The price of our choice is twofold. First, M6 as we report it is not the off-the-shelf Entropia `-bgen` output but our own token-replay scoring of its genuine `bsgen` sampler, so it is an adaptation, not a drop-in published baseline. Second, that scoring is the conformance core of our metric and of R1, a shared-ruler effect we flag in Sect. 6.9.

5.5 Deviations from the formal method

Three deviations are worth recording. (i) As noted in Sect. 4.6, the N-gram model is rebuilt each of the K iterations rather than once; this is a performance artifact,

Table 1. Benchmark datasets. TLRA $= 1 - |V|/|L|$.

ID	Log	Cases	Events	Variants	Avg. len	TLRA
D1	Sepsis [17]	1,050	15,214	846	14.5	0.19
D2	BPI 2013 Incidents	7,554	65,533	1,511	8.7	0.80
D3	BPI 2017	31,509	1,202,267	15,930	38.2	0.49
D4	BPI 2018	43,809	2,514,266	28,457	57.4	0.35
D5	BPI 2019	251,734	1,595,923	11,973	6.3	0.95

not a semantic one. (ii) The strict reading counts a shadow trace as accepted when PM4Py's token replay flags it as perfectly fitting (`trace_is_fit`). That flag is a fast but approximate test of language membership, which is properly decided by a cost-zero alignment. On nets with invisible (silent) or duplicate-label transitions the two can disagree, so our acceptance number is a proxy and should be spot-checked against alignments on such nets. (iii) The data-driven length cap of v2.6 never binds on D1/D2 (`truncated_traces`=0), so the v2.6 `log` mode (M1f) is virtually identical to v2.5 (M1e) there; the cap, and hence any difference, will first matter on deeper-trace datasets (D3–D4).

6 Evaluation

This is an algorithmic, quantitative contribution, so we evaluate experimentally. We measure how well each metric agrees with an empirical generalization ground truth across discovery configurations and datasets, and we attribute the proposed metric's behavior to specific design decisions through an ablation lineage.

6.1 Experimental setup

Datasets. From a profiled catalog of 20+ public logs we selected five (Table 1) spanning trace depth, variant diversity, scale, and, most deliberately, trace-level representativeness (TLRA, ranging 0.19–0.95), the axis shown to govern the accuracy of generalization estimation [13]. The same evaluation battery runs on every dataset. We present D1 (Sepsis) in full as the primary case, carrying the cross-paradigm comparison and its figures; D2 (BPI 2013 Incidents) runs the same methods and corroborates the version study, the litmus, and the acceptance reading (Sects. 6.3–6.6). The three larger logs D3–D5 extend the study to greater scale and are in progress on dedicated high-memory hardware (Sect. 7).

Discovery configurations. Eight miners span the construct's poles: a filtered trace model (top-50 variants by frequency; the full trace model is intractable to replay on variant-rich logs), Alpha and Alpha+ [3], Heuristics default and strict [23], Inductive strict and infrequent [15,16], and a flower model. Models are discovered once per (dataset, miner) cell and shared by all methods.

Table 2. ShadowGen version lineage and benchmark roles.

Version	Role	Key change
v1	M1a	1-gram directly-follows generator, Good–Turing mutation
v2.1	M1b (N=3), M1c (N=6)	N-gram contexts, Katz backoff, $\ln(f+1)$ weights
v2.2–v2.3	(none)	structural term added, then retired; deduplication; context-aware p_{end}
v2.4	M1d	variant compression, memoization, iterative walker; uniform mutation
v2.5	M1e	Katz-consistent mutation proposal; integrity counters
v2.6	M1f (`log`), M1g (`mle`)	acceptance reading; data-driven length cap; sampling-mode switch

Three representative versions are carried into the evaluation: **v1** (M1a, the simplest baseline), **v2.1** at N=6 (M1c, full N-gram context), and **v2.6-mle** (M1g, the final metric). The intermediate versions v2.4/v2.5 isolate the two fixes discussed in Sect. 6.5.

Methods and ground truth. The metric family spans seven development versions M1a–M1g (Table 2), of which we feature three in the evaluation: v1, v2.1 (N=6), and the final v2.6-mle. External baselines are M2–M8 (Sect. 3). Reference metrics are R1 (variant-based 5-fold CV fitness, 3 shuffles), R2 (leave-one-variant-out), and R3 (replay of uniformly random traces, a floor). An acceptance-based variant, R1-accept, measures the fraction of held-out traces replayed perfectly and validates the acceptance reading.

Protocol and agreement criteria. Each cell is evaluated with its method's stochasticity protocol (the metric family: 5 iterations of 1,000 shadow traces, mean ± std; seed 42). Agreement with the ground truth is reported on *four criteria*: Pearson (calibration shape), Spearman and Kendall's τ_b (ordering), mean absolute error (MAE, absolute calibration), and discriminative spread (max−min over the six non-degenerate miners). All four are computed over those six miners; the two poles are excluded and reported separately as litmus values. Kendall's τ_b is the fraction of model pairs ordered the same way by the metric and the ground truth, tie-corrected; with six miners a perfect ordering has exact permutation probability $1/6! \approx 0.0014$, which keeps the ranking claims interpretable despite the small sample. Rank correlation alone is too weak a test: as Sect. 6.3 shows, a random-replay baseline also reaches Spearman and Kendall 1.0. The context order was fixed at N=6 via a mutation-rate sweep on BPI 2017 (the realized mutation rate peaks at N=6 and declines beyond, as backoff increasingly rejects sparse contexts; Fig. 3), and N=6 is held fixed across all datasets so the metric is never tuned per dataset.

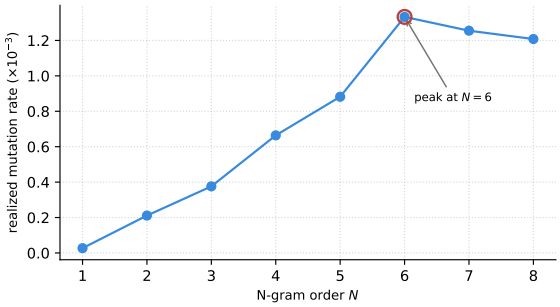

Fig. 3. Realized mutation rate vs. N-gram order on BPI 2017 (the one calibration of the metric). The rate climbs to a peak at N=6 and declines beyond, as Katz backoff increasingly rejects over-sparse high-order contexts; N=6 is then held fixed on every dataset.

Compute budget. [Exact budget still in progress.] Every method is granted the same time budget per (dataset, miner) cell. A cell that does not return within the budget is recorded as a sentinel (-1, "exceeds budget") rather than dropped, and a method is not re-run after a timeout. This is a deliberate, uniform protocol rule with two purposes: it keeps the comparison affordable at scale, and, since the working metrics complete a cell in seconds, it turns runtime into a first-class outcome, so that a method's inability to finish is reported as a result rather than silently omitted. Sentinels are excluded from the agreement statistics and shown as distinct cells (not low scores) in the heatmaps.

6.2 Feasibility results

Under the compute budget, two published metrics return no score on any cell, which we report as a result. Anti-alignments (M4) ran on the small demo log bundled with its implementation (10 traces, 53 ms), confirming the install works, but produced no score on any miner cell of the 1,050-case Sepsis log: on Alpha+ its Gurobi ILP ran 13.4 h without returning, while on the remaining miners the solver failed outright within seconds. Pattern-based generalization (M8) likewise yielded nothing on every model: it returned no result within its timeout, or failed to initialize its display server. These are not one-off accidents but a reproducible finding: against seconds per cell for every working metric, methods that cannot finish on a small real log are impractical, and the gap widens with dataset size. A third method, entropic relevance (M3), is degraded rather than budget-exceeding (Sect. 5.4) and is excluded from comparison. Of seven published baselines, four (M2, M5, M6, M7) currently produce model-discriminating scores within budget on a small real-life log. AVATAR (M5) is expected to exceed the budget from D3 upward, where GAN training dominates. [insert D3–D5 here]

Table 3. D1 Sepsis: scores per miner, mean ± std where applicable (seven miners of the external comparison). ‡model fitness < 0.5 on the log. †infeasible (no score on any cell).

Method	Alpha‡	Alpha+	Heur.	Heur.S	Ind.Inf	Ind.S	Flower
v1 (ours)	.265±.003	.606±.004	.865±.002	.893±.002	.911±.000	.975±.001	1.00
v2.6-mle (ours)	.272±.004	.759±.005	.879±.002	.917±.002	.972±.002	.984±.002	1.00
M2 PM4Py	.913	.919	.841	.900	.880	.903	.913
M5 AVATAR	.340±.020	.562±.008	.746±.001	.704±.002	.751±.002	.535±.009	.397±.007
M6 Bootstrap*	.252±.005	.783±.010	.897±.004	.931±.003	.976±.002	.994±.001	1.00
M7 SpeciAL	.799	.750	1.00	.998	.750	.744	.803
M4 Anti-align.†	*infeasible: no score on any cell (Sect. 6.2)*						
M8 Pattern†	*infeasible: no score on any cell*						
R1 5-fold CV	.275±.001	.829±.002	.902±.005	.933±.001	.985±.002	1.00	1.00
R2 LOVO	.306±.075	.775±.177	.870±.051	.917±.044	.981±.053	1.00	1.00
R3 Random	.278±.004	.382±.004	.502±.005	.621±.003	.693±.001	.767±.002	1.00

*M6 scores the genuine `bsgen` bootstrap sample by token-replay fitness (a graded model–system recall, the construct the bootstrap paper defines [18]), not the Entropia `-bgen` precision/recall F-measure (Sect. 5.4). †M4 ran 13.4 h on Alpha+ and failed in seconds on the rest; M8 timed out or failed to initialize.

Table 4. Agreement with R1 on D1 (six real miners) and the flower litmus. τ_b is Kendall's tau-b.

Method	Pearson	Spearman	τ_b	MAE	Spread	Flower
v1 (ours)	0.96	1.00	1.00	0.068	0.71	1.00
v2.6-mle (ours)	1.00	1.00	1.00	0.023	0.71	1.00
M2 PM4Py	−0.37	−0.43	−0.20	0.17	0.08	0.91
M5 AVATAR	0.81	0.37	0.33	0.24	0.41	0.40
M6 Bootstrap*	1.00	1.00	1.00	0.015	0.74	1.00
M7 SpeciAL	0.12	−0.46	−0.41	0.21	0.26	0.80
R3 Random	0.83	1.00	1.00	0.28	0.49	1.00

6.3 Cross-paradigm comparison on D1

Tables 3–4 compare paradigms against the cross-validation ground truth. Four findings stand out.

1. Generative and resampling paradigms track held-out fitness; structural and diversity paradigms do not. The most widely available metric (M2) correlates negatively with the ground truth, compresses seven structurally very different models into a spread of 0.08, and ranks the pathological Alpha model highest. This is direct evidence that visit-frequency counting does not measure held-out generalization.
2. Rank correlation alone is a low bar. The random-replay floor R3 attains Spearman and τ_b of 1.0, because the Sepsis miners differ so strongly in permissiveness that almost any test orders them correctly. Validation must rest on the full set of criteria, which is where v2.6-mle and bootstrap (MAE 0.023/0.015, spreads 0.71/0.74) separate from R3 (MAE 0.28, spread 0.49).
3. The flower litmus sorts the field. Pure-acceptance metrics (the M1 family, M6, R1–R3) score the flower model 1.0, as the construct requires; M2, M7,

Table 5. Robustness to the cross-validation reference on D1 (six real miners): agreement against both R1 (5-fold) and R2 (leave-one-variant-out, 50/846 variants sampled). The qualitative verdict is identical under either reference.

	vs R1 (5-fold)			vs R2 (LOVO)		
Method	Pear.	τ_b	MAE	Pear.	τ_b	MAE
v2.6-mle (ours)	1.00	1.00	0.023	1.00	1.00	0.014
M6 Bootstrap*	1.00	1.00	0.015	1.00	1.00	0.019
M2 PM4Py	−0.37	−0.20	0.17	−0.37	−0.20	0.17
M5 AVATAR	0.81	0.33	0.24	0.80	0.33	0.21
M7 SpeciAL	0.12	−0.41	0.21	0.10	−0.41	0.20

and especially M5 (0.40) reveal precision/structure contamination. That M6 lands in the first group is independent corroboration, not coincidence: the bootstrap metric is itself defined as model–system recall [18], so a separate, published generalization measure also rates the flower model maximally general. M5's low flower score, by contrast, follows from its construct (a harmonic mean of fitness and precision [22], i.e. an F-measure), which also explains its poor rank fidelity on the real miners.

4. Bootstrap is the closest competitor, with a caveat. M6 matches our calibration, but it scores with token-replay fitness, the same conformance engine as R1 and our own metric; part of its tight agreement is a shared-ruler effect rather than independent corroboration (Sect. 6.9).

The agreement is robust to the choice of cross-validation reference (Table 5). Against the finer-grained leave-one-variant-out (R2), ShadowGen's ranking is identical (Kendall τ_b=1.0) and its calibration is unchanged on D1 (MAE 0.014); on D2 the ranking again holds while calibration stays strong (Pearson 0.97, MAE 0.048). The verdicts on every baseline are the same under R1 and R2.

The litmus failures also replicate on D2: PM4Py scores the flower model 0.88, AVATAR 0.74, and SpeciAL 0.94, all below the 1.0 the construct requires, while the M1 family and the references stay at 1.0.

Figure 4 plots each metric against the cross-validation ground truth directly; the raw per-cell scores are in Table 3.

6.4 What contaminates a generalization score: a controlled M6 study

Bootstrap generalization is published as a generalization metric in its own right [18], so it is worth asking why the number a user typically reads off it (the Entropia `-bgen` F1 of model–system precision and recall) fails the flower litmus. Because one bred `bsgen` sample can be scored several ways, we hold the sampler fixed and vary only the scoring (Table 6).

The two readings that include precision fail the litmus (flower 0.18, 0.10) and miscalibrate badly (MAE 0.59, 0.68); the two recall-only readings pass and

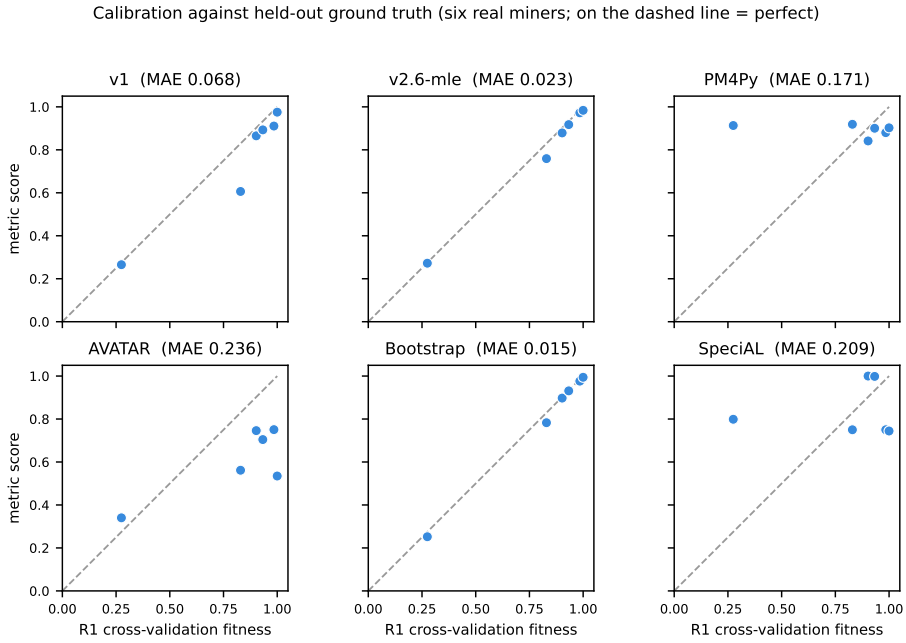

Fig. 4. Per-method calibration against the cross-validation ground truth on D1 (six real miners; each point is one miner, dashed line $y{=}x$). The generative and resampling metrics (v1, v2.6-mle, Bootstrap) sit on the line; PM4Py forms a flat high band that ignores the ground-truth gradient: it rates the pathological Alpha model (at R1$\approx$ 0.27) as 0.91; AVATAR and SpeciAL scatter off the line. Reading the vertical axis alone, a metric that spreads its six miners discriminates between them, whereas PM4Py compresses them into a narrow band.

track R1. The F1 still reaches Pearson 0.87, yet its MAE 0.59 and flower 0.18 expose it, which is why the protocol uses four criteria and not one. So it is the precision component, not the bootstrap sampler, that throws M6 off, and this is why we report M6 by token-replay fitness, faithful to the paper's recall construct (Sect. 5.4).

6.5 Version study: the M1 family on D1 and D2

Tables 7–8 report the three representative versions. The shape is telling: adding N-gram context (v2.1) makes calibration worse than the 1-gram baseline, the next versions only claw back to it, and the metric beats v1 for the first time at v2.6-mle, when frequency-faithful sampling replaces the log-damping. The win is the sampling fix, not the context, which only earns its keep once paired with it and is predicted to matter more on denser logs (Finding 4 below).

1. Context alone is not enough; the v2.6 fixes are decisive. The 1-gram baseline v1 is already strong on these logs (D1 MAE 0.068, D2 0.036). Adding full

Table 6. The same `bsgen` bootstrap sample on D1, scored four ways, vs R1 (six real miners). Only the readings free of precision pass the flower litmus.

Reading of the bootstrap sample	Pearson	τ_b	MAE	Flower	litmus
Entropia `-bgen` F1(prec, rec)	0.87	0.20	0.59	0.18	fail
its precision component	0.82	0.20	0.68	0.10	fail
its **recall** component	0.98	0.73	0.11	1.00	pass
our adaptation (token-replay)	1.00	1.00	0.015	1.00	pass

Table 7. M1 family, mean ± std over 5 iterations (seed 42), with both poles. Tr. = filtered trace model.

	Trace	Alpha	Alpha+	Heur.	Heur.S	Ind.Inf	Ind.S	Flower
D1 Sepsis								
R1	.641	.275	.829	.902	.933	.985	1.00	1.00
v1	.562±.003	.265±.003	.606±.004	.865±.002	.893±.002	.911±.000	.975±.001	1.00
v2.1 (N=6)	.517±.005	.297±.003	.627±.006	.836±.001	.854±.001	.916±.001	.956±.002	1.00
v2.6-mle	.569±.003	.272±.004	.759±.005	.879±.002	.917±.002	.972±.002	.984±.002	1.00
D2 BPI 2013 Incidents								
R1	.653	.215	.698	.996	.998	.988	1.00	1.00
v1	.634±.003	.313±.003	.587±.002	.997±.000	.999±.000	.996±.000	1.00	1.00
v2.1 (N=6)	.605±.002	.367±.007	.540±.005	.969±.001	.978±.001	.975±.001	1.00	1.00
v2.6-mle	.600±.003	.257±.005	.630±.008	.994±.000	.998±.000	.991±.001	1.00	1.00

N-gram context with the original log-weighting and uniform mutation (v2.1) worsens the quality (D1 0.068 → 0.080; D2 0.036 → 0.062). Only v2.6-mle, which pairs the context with a Katz-consistent mutation proposal and frequency-faithful (`mle`) sampling, delivers the gain: D1 MAE 0.080 → 0.023, D2 0.062 → 0.019. (The intermediate versions v2.4/v2.5 of Table 2 isolate the two fixes individually; per-version detail is omitted for brevity.)

2. v2.6-mle corrects a ranking error. On D2 the ground truth places Heuristics above Inductive-infrequent; v2.1 and every log-weighted version invert this pair (Spearman 0.943), and v2.6-mle restores it (Spearman 1.000). Robustness was checked across seeds (D1: four seeds; D2: two seeds; std of Pearson and MAE ≤ 0.001).
3. The poles behave only under the strict reading. Every version scores the flower model exactly 1.0. Under graded fitness, however, the filtered trace model is not the lowest-scoring model: token replay grants it partial credit (0.52–0.64), and the pathological Alpha model scores lower still (on D1, v2.6-mle gives Alpha 0.27 against the trace model's 0.57). Even the ground truth assigns the memorizing model 0.64–0.65. Only the strict acceptance reading (Sect. 6.6) places the trace model at a literal 0 and so anchors the low pole, which is one reason that reading matters.
4. One honest caveat: on D2's four-activity alphabet, v1 nearly matches v2.6-mle, so deep context earns its keep on alphabet-rich logs, not trivial ones. We register the prediction that the versions separate on dense logs (D3, where N=6 resolves 80% of decisions at top order) as a falsifiable test for the larger-log evaluation.

Table 8. M1-family agreement with R1 (six real miners), three representative versions.

	D1 Sepsis				D2 BPI 2013			
Version	Pear.	Spear.	MAE	Spr.	Pear.	Spear.	MAE	Spr.
v1	0.958	1.000	0.068	0.710	0.979	1.000	0.036	0.687
v2.1 (N=6)	0.966	1.000	0.080	0.659	0.954	0.943	0.062	0.633
v2.6-mle	**0.996**	**1.000**	**0.023**	**0.711**	**0.994**	**1.000**	**0.019**	**0.743**

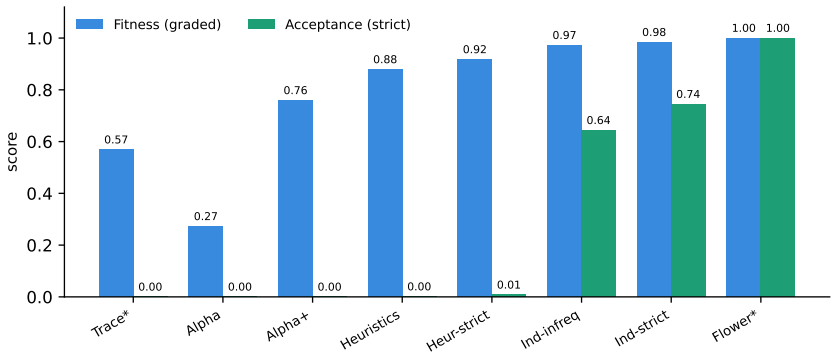

Fig. 5. Two readings of the shadow log on D1 Sepsis (v2.6-mle): graded fitness (partial credit) vs. strict acceptance (perfect-replay fraction). Acceptance gives the construct's clean poles (Trace 0.00, Flower 1.00), and the gap between the bars exposes partial-credit inflation: e.g. Alpha+ scores 0.76 fitness but 0.00 acceptance, and the trace model 0.57 fitness but 0.00 acceptance.

6.6 The acceptance reading

Gen_{accept} reports the fraction of shadow traces replayed perfectly, the strict membership-shaped reading. It is best read not as a competing quality score (it is bimodal and rank-uninformative, see below) but as a diagnostic that decomposes the graded one: Gen_{shadow} is real acceptance plus partial credit, and the per-model gap between the two says how much of a score is real. Acceptance values must be validated against an acceptance-based ground truth (R1-accept: the fraction of held-out real traces replayed perfectly). Against the matching ground truth, v2.6-mle achieves Pearson 0.998/0.997 and MAE 0.076/0.021 on D1/D2; the D1 detail below carries the argument, and the same agreement holds on D2 (Table 9, Fig. 5).

Three observations. The poles anchor exactly: trace model 0.000, flower model 1.000, in both metric and ground truth, on both datasets. These are the clean 0/1 bounds that graded fitness cannot provide. Near-zero acceptances are true model properties, not metric failures: on D1 the ground truth itself shows Alpha, Alpha+, and both Heuristics models strictly accepting (almost) no unseen real traces (e.g., Alpha+ 0.000, Heuristics 0.028); only the Inductive family

Table 9. Acceptance reading on D1 (v2.6-mle Gen_{accept}) vs the acceptance ground truth (R1-accept).

	Trace	Alpha	Alpha+	Heur.	Heur.S	Ind.Inf	Ind.S	Flower
Gen_{accept} (v2.6-mle)	.000	.000	.000	.001	.008	.644	.744	1.00
R1-accept	.000	.000	.000	.028	.043	.783	.996	1.00

genuinely accepts novel behavior. Alpha+'s respectable graded score (0.83 under R1) is thus pure partial credit, which the acceptance reading exposes in one line, and the per-model gap between Gen_{shadow} and Gen_{accept} is itself diagnostic. Caveats: strict acceptance is bimodal and sensitive to net soundness (which is why it accompanies rather than replaces the graded score), rank correlation is uninformative under mass ties at zero (use Pearson/MAE), and Gen_{accept} underestimates R1-accept in the mid-range on D1 because shadow traces (containing novel recombinations and mutations) are deliberately harder than held-out real traces.

6.7 Runtime

The metric evaluates one cell ($5 \times 1{,}000$ traces) in 2–7 seconds, independent of version; the v2.6 additions cost $\leq 6\%$ over the earlier versions. Figure 6 sets accuracy against time: bootstrap is $\sim$ 11 s, the R1 ground truth is minutes per dataset, and leave-one-variant-out and AVATAR are hours. Among methods that agree with the ground truth, ours is the fastest, and it stays defined where the ground truth is not: it scores a model artifact in hand, without rediscovery (Sect. 4.6).

6.8 Discussion

On the two datasets evaluated, the methods that confront a model with resampled or synthesized behavior (our metric, bootstrap) agree with the cross-validation ground truth in both rank and absolute value, whereas methods that inspect structure or aggregate diversity (PM4Py, SpeciAL) do not. This supports the project's core hypothesis: generalization is a claim about unseen behavior and is therefore best measured by probing models with plausible unseen behavior, instead of analysing the existing structure. Our metric reaches this agreement at interactive runtime with interpretable internals (per-context novelty, mutation flags, the openness profile, the acceptance reading) that neural and bootstrap aggregates do not expose. The honest claim is therefore not "best metric" but "matches the strongest baseline's calibration while probing genuinely unseen behavior that resampling cannot reach, and exposing why a model scored as it did, at comparable cost and with no external toolchain."

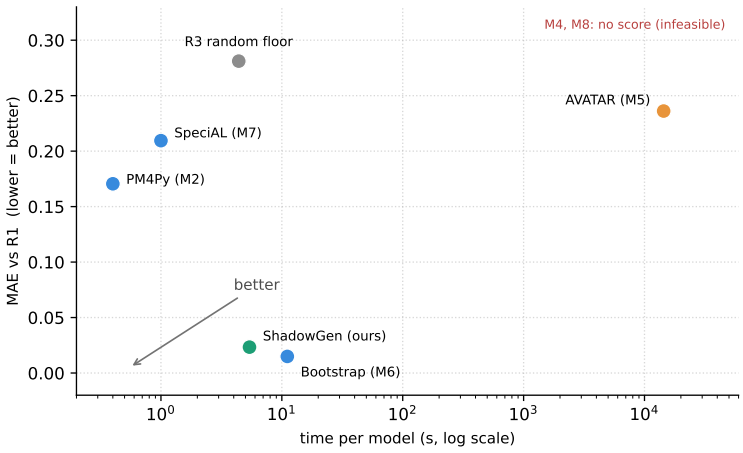

Fig. 6. Speed against accuracy on D1: time per model (log scale) versus mean absolute error to R1; lower-left is better. PM4Py is fast but inaccurate, AVATAR is both slow and inaccurate, while ShadowGen and bootstrap sit at low error and interactive speed. R1 (the ground truth) costs about two minutes and cannot score a model in hand; anti-alignments (M4) and pattern-based (M8) return no score at all (Sect. 6.2).

6.9 Threats to validity

Breadth: external methods are compared on one dataset and the version study on two; correlations over six miners are fragile, and dataset-specific structure plausibly favors particular methods. D3–D5 are designed to stress different regimes, and the registered D3 ablation-separation prediction makes the larger-log evaluation a test rather than a formality.

Shared measurement core: the metric, R1, R3, and the adapted M6 all score via token replay, so part of their mutual agreement may reflect a shared fitness notion rather than shared generalization semantics. R1's per-fold rediscovery limits but does not eliminate this, and the acceptance reading was already validated against a separately computed ground truth (R1-accept).

Ground truth and representativeness: our validation rests on agreement with variant-based hold-out (R1/R2). We regard this as the strongest available empirical reference for generalization, because its test traces are real held-out behavior and so are provably valid, where a shadow trace is only plausible by construction. The close agreement between the two is therefore evidence that the shadow log behaves like genuine future behavior. ShadowGen also reaches this agreement while scoring a model in hand, which R1 cannot: R1 evaluates a discovery algorithm by rediscovering on log subsets, not a model artifact. Both are computed from the log, so both are only as faithful to the true system as the log is representative of it. We treat that as a responsibility of the data-gathering stage rather than of the metric: any analysis built on a log, including a model designed by hand, is bounded in validity by how representatively the log samples the un-

derlying process. Agreement with R1 should therefore be read as agreement with the best available empirical reference, not with true-system generalization in an absolute sense. (We predict that ShadowGen's MAE to R1 grows as representativeness falls; D3–D5, at TLRA 0.49–0.95, will test this.)

Generator validity: the premise that the shadow log resembles valid future behavior rests on construction, since traces are generated from the log's own variable-order statistics with calibrated Good–Turing novelty, so every shadow trace is a plausible continuation by design. An empirical cross-check (for example, measuring overlap between shadow traces and held-out variants) is a possible future strengthening but is not required for the constructive argument.

Adapted and incomplete baselines: M6 uses token-replay fitness in place of the Entropia precision/recall F-measure (Sect. 5.4); M3 awaits a per-model stochastic-automaton conversion; M5 has two of 3–5 planned sampling runs; R2 sampled 50 of 846 variants on D1.

Calibration provenance: N=6 was selected on one log via a proxy criterion (mutation rate), and the mutated-trace share varies strongly across logs ($\sim 4\%$ on BPI 2017 vs. $\sim 50\%$ on Sepsis), so proposal-distribution effects are dataset-dependent by construction.

Implementation proxy: the strict acceptance reading decides whether a shadow trace is in the model's language using PM4Py's token-replay flag (`trace_is_fit`), a fast approximation that can disagree with a cost-zero alignment on nets with silent or duplicate-label transitions (Sect. 5.5).

7 Conclusion

Contribution and findings. We presented ShadowGen, a generative N-gram metric for process-model generalization, together with a strict construct definition it implements and a benchmark that validates generalization metrics against variant-based cross-validation under a four-criterion protocol with both construct poles included. The central result is that ShadowGen tracks the empirical ground truth: it reproduces the cross-validation ranking and is calibrated to it (Pearson 0.996/0.994, MAE 0.023/0.019) at seconds per model, and it does so while scoring a model in hand, where the cross-validation reference cannot. Its calibration is attributable to two measured design changes. The benchmark adds findings of independent value: the field's most accessible metric (PM4Py) is anti-correlated with empirical generalization on Sepsis; two published metrics are computationally infeasible on a small real log; rank correlation alone is too weak a validation criterion; and a strict acceptance reading, validated against its own ground truth, anchors the construct's poles exactly while exposing partial-credit inflation in graded scores.

Limitations. The metric and much of its cross-validation reference share a token-replay core, so part of their agreement reflects a shared conformance notion rather than shared generalization semantics; the context order N=6 is calibrated on a single log via a proxy; several baselines are adapted, degraded, or

incomplete; and the central premise, that the shadow log resembles valid future behavior, rests on construction and has not yet been directly tested.

Future work. The natural next steps are a direct test of the generator premise, by measuring the overlap between shadow traces and held-out behavior, and restoring entropic relevance through a per-model stochastic-automaton conversion. The metric itself is feature-frozen, with the `mle` mode as the default and the name ShadowGen reflecting its purely generative design.

Declaration on the Use of AI-Based Tools

In preparing this work, we used AI-based assistants (large-language-model and coding tools) for source-code drafting, figure generation, and language editing; all content was reviewed and verified by the authors, who take full responsibility for it.

References

1. van der Aalst, W.M.P.: Process Mining: Data Science in Action. Springer, Heidelberg, 2nd edn. (2016). https://doi.org/10.1007/978-3-662-49851-4
2. van der Aalst, W.M.P.: Relating process models and event logs — 21 conformance propositions. In: Proc. Int. Workshop on Algorithms & Theories for the Analysis of Event Data (ATAED 2018). CEUR Workshop Proceedings, vol. 2115, pp. 56–74 (2018)
3. van der Aalst, W.M.P., Weijters, A.J.M.M., Maruster, L.: Workflow mining: Discovering process models from event logs. IEEE Transactions on Knowledge and Data Engineering **16**(9), 1128–1142 (2004). https://doi.org/10.1109/TKDE.2004.47
4. Alkhammash, H., Polyvyanyy, A., Moffat, A., García-Bañuelos, L.: Entropic relevance: A mechanism for measuring stochastic process models discovered from event data. Information Systems **107**, 101922 (2022). https://doi.org/10.1016/j.is.2021.101922
5. Augusto, A., Conforti, R., Dumas, M., La Rosa, M., Maggi, F.M., Marrella, A., Mecella, M., Soo, A.: Automated discovery of process models from event logs: Review and benchmark. IEEE Transactions on Knowledge and Data Engineering **31**(4), 686–705 (2019). https://doi.org/10.1109/TKDE.2018.2841877
6. Berti, A., van Zelst, S.J., van der Aalst, W.M.P.: Process mining for Python (PM4Py): Bridging the gap between process- and data science. In: Proceedings of the ICPM Demo Track 2019. CEUR Workshop Proceedings, vol. 2374, pp. 13–16 (2019)
7. Buijs, J.C.A.M., van Dongen, B.F., van der Aalst, W.M.P.: Quality dimensions in process discovery: The importance of fitness, precision, generalization and simplicity. International Journal of Cooperative Information Systems **23**(1), 1440001 (2014). https://doi.org/10.1142/S0218843014400012
8. van Dongen, B.F., Carmona, J., Chatain, T.: A unified approach for measuring precision and generalization based on anti-alignments. In: Business Process Management (BPM 2016). LNCS, vol. 9850, pp. 39–56. Springer (2016). https://doi.org/10.1007/978-3-319-45348-4_3

9. Gale, W.A., Sampson, G.: Good–Turing frequency estimation without tears. Journal of Quantitative Linguistics **2**(3), 217–237 (1995). https://doi.org/10.1080/09296179508590051
10. Janssenswillen, G., Depaire, B.: Towards confirmatory process discovery: Making assertions about the underlying system. Business & Information Systems Engineering **61**(6), 713–728 (2019). https://doi.org/10.1007/s12599-019-00610-6
11. Janssenswillen, G., Donders, N., Jouck, T., Depaire, B.: A comparative study of existing quality measures for process discovery. Information Systems **71**, 1–15 (2017). https://doi.org/10.1016/j.is.2017.06.002
12. Kabierski, M., Richter, M., Weidlich, M.: Addressing the log representativeness problem using species discovery. In: Proceedings of the 5th International Conference on Process Mining (ICPM 2023). pp. 65–72. IEEE (2023). https://doi.org/10.1109/ICPM60904.2023.10271952
13. Karunaratne, A., Polyvyanyy, A., Moffat, A.: The role of log representativeness in estimating generalization in process mining. In: Proceedings of the 6th International Conference on Process Mining (ICPM 2024). IEEE (2024)
14. Katz, S.M.: Estimation of probabilities from sparse data for the language model component of a speech recognizer. IEEE Transactions on Acoustics, Speech, and Signal Processing **35**(3), 400–401 (1987). https://doi.org/10.1109/TASSP.1987.1165125
15. Leemans, S.J.J., Fahland, D., van der Aalst, W.M.P.: Discovering block-structured process models from event logs — A constructive approach. In: Application and Theory of Petri Nets and Concurrency (PETRI NETS 2013). LNCS, vol. 7927, pp. 311–329. Springer (2013). https://doi.org/10.1007/978-3-642-38697-8_17
16. Leemans, S.J.J., Fahland, D., van der Aalst, W.M.P.: Discovering block-structured process models from event logs containing infrequent behaviour. In: Business Process Management Workshops (BPM 2013). LNBIP, vol. 171, pp. 66–78. Springer (2014). https://doi.org/10.1007/978-3-319-06257-0_6
17. Mannhardt, F.: Sepsis cases — event log (2016). https://doi.org/10.4121/uuid:915d2bfb-7e84-49ad-a286-dc35f063a460
18. Polyvyanyy, A., Moffat, A., García-Bañuelos, L.: Bootstrapping generalization of process models discovered from event data. In: Advanced Information Systems Engineering (CAiSE 2022). LNCS, vol. 13295, pp. 36–54. Springer (2022). https://doi.org/10.1007/978-3-031-07472-1_3
19. Reißner, D., Armas-Cervantes, A., Conforti, R., Dumas, M., Fahland, D., La Rosa, M.: Scalable alignment of process models and event logs: An approach based on automata and S-components. Information Systems **94**, 101561 (2020). https://doi.org/10.1016/j.is.2020.101561
20. Rozinat, A., van der Aalst, W.M.P.: Conformance checking of processes based on monitoring real behavior. Information Systems **33**(1), 64–95 (2008). https://doi.org/10.1016/j.is.2007.07.001
21. Syring, A.F., Tax, N., van der Aalst, W.M.P.: Evaluating conformance measures in process mining using conformance propositions. In: Transactions on Petri Nets and Other Models of Concurrency XIV, LNCS, vol. 11790, pp. 192–221. Springer (2019). https://doi.org/10.1007/978-3-662-60651-3_8
22. Theis, J., Darabi, H.: Adversarial system variant approximation to quantify process model generalization. IEEE Access **8**, 194410–194427 (2020). https://doi.org/10.1109/ACCESS.2020.3033450
23. Weijters, A.J.M.M., van der Aalst, W.M.P., de Medeiros, A.K.A.: Process mining with the HeuristicsMiner algorithm. Tech. Rep. WP 166, Eindhoven University of Technology (2006)